

Security Audit

Report for Flap Protocol V5

Date: December 25, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About Target Contracts	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Security Issues	2
1.3.2 Additional Recommendation	3
1.4 Security Model	3
Chapter 2 Findings	4
2.1 Security Issue	5
2.1.1 Potential skipping of token distribution in the function <code>distributeTokens()</code>	5
2.1.2 Lack of token transfer logic in function <code>claimTokens()</code>	6
2.1.3 Significant price deviation between <code>bonding curve</code> and DEX launch price	6
2.1.4 Lack of relative privileged function implementation in contract <code>Portal</code>	7
2.1.5 Potential DoS due to the improper native token swap logic	10
2.1.6 Potential DoS due to incorrect validation in function <code>_participate()</code>	11
2.1.7 Incorrect fee calculation in the function <code>_previewBondingCurveBuy()</code>	12
2.1.8 Lack of native token refund mechanism	12
2.1.9 Lack of check in function <code>stageNewTokenV5()</code>	13
2.1.10 Potential DoS while handling non-compliant ERC20 tokens	14
2.1.11 Potential overcharge in the function <code>createSale()</code>	14
2.1.12 Potential permanent lock of residual tax tokens in function <code>_liquidateTax()</code>	15
2.1.13 Potential sandwich attacks due to the lack of slippage protections	15
2.2 Recommendation	16
2.2.1 Add allowance revocation logic	16
2.2.2 Revise the validation order in the function <code>_bondingCurveSell()</code>	17
2.2.3 Remove redundant code	17
2.2.4 Fix typo	18
2.2.5 Add a tax validation check in the function <code>setTaxConfig()</code>	19
2.3 Note	19
2.3.1 Potential centralization risks	19
2.3.2 Disabled two-step token creation and permitted token name/symbol duplication	20
2.3.3 Dividend withdrawal failure handling mechanism	20

Report Manifest

Item	Description
Client	Flap.sh
Target	Flap Protocol V5

Version History

Version	Date	Description
1.0	December 25, 2025	First release

Signature



Digitally signed by: BlockSec (<https://blocksec.com>)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2025-12-26 08:37:37+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository ¹ of Flap Protocol V5 of Flap.sh.

Flap Protocol is a decentralized token launch platform built on bonding curve mechanics. Users can launch tokens on customizable bonding curves, with automatic migration to DEX when circulating supply reaches predefined thresholds. The protocol uses a modular architecture centered around the Portal contract, which dispatches calls to specialized modules via delegatecall: PortalTokenLauncher handles token creation, PortalTradeV2 manages bonding curve trading, and PortalUniV2Migrator/PortalUniV3Migrator execute DEX migrations. Built on top is FlapSale, a presale mechanism that enables fair token distribution with price guarantees. The protocol supports multiple token types including standard ERC20 tokens and advanced tax tokens (FlapTaxTokenV2) with features like deflation, dividends, and liquidity provision. All modules inherit from PortalBase to ensure consistent storage layout, enabling seamless inter-module communication while maintaining security and upgradability.

Note this audit only focuses on the smart contracts in the following directories/files:

- src/

Other files are not within the scope of the audit. Additionally, all dependencies of the smart contracts within the audit scope are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version ([Version 1](#)), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version ([Version 0](#)), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
Flap Protocol V5	Version 1	c1395fd7af7f4b957d8a82d49ab22c6e71131ed6
	Version 2	ad5e73b0742d3e7f58abd3cddb1a3c07de006af0

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on,

¹<https://github.com/flap-sh/Flap-contract-for-audit>

the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

- * Access control
- * Permission management
- * Whitelist and blacklist mechanisms
- * Initialization consistency
- * Improper use of the proxy system
- * Reentrancy
- * Denial of Service (DoS)
- * Untrusted external call and control flow
- * Exception handling
- * Data handling and flow
- * Events operation
- * Error-prone randomness
- * Oracle security
- * Business logic correctness
- * Semantic and functional consistency

- * Emergency mechanism
- * Economic and incentive impact

1.3.2 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology ² and Common Weakness Enumeration ³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we found **thirteen** potential security issues. Besides, we have **five** recommendations and **three** notes.

- High Risk: 3
- Medium Risk: 3
- Low Risk: 7
- Recommendation: 5
- Note: 3

ID	Severity	Description	Category	Status
1	High	Potential skipping of token distribution in the function <code>distributeTokens()</code>	Security Issue	Fixed
2	High	Lack of token transfer logic in function <code>claimTokens()</code>	Security Issue	Fixed
3	High	Significant price deviation between <code>bonding curve</code> and DEX launch price	Security Issue	Fixed
4	Medium	Lack of relative privileged function implementation in contract <code>Portal</code>	Security Issue	Confirmed
5	Medium	Potential DoS due to the improper native token swap logic	Security Issue	Fixed
6	Medium	Potential DoS due to incorrect validation in function <code>_participate()</code>	Security Issue	Fixed
7	Low	Incorrect fee calculation in the function <code>_previewBondingCurveBuy()</code>	Security Issue	Fixed
8	Low	Lack of native token refund mechanism	Security Issue	Confirmed
9	Low	Lack of check in function <code>stageNewTokenV5()</code>	Security Issue	Fixed
10	Low	Potential DoS while handling non-compliant ERC20 tokens	Security Issue	Confirmed
11	Low	Potential overcharge in the function <code>createSale()</code>	Security Issue	Confirmed
12	Low	Potential permanent lock of residual tax tokens in function <code>_liquidateTax()</code>	Security Issue	Fixed
13	Low	Potential sandwich attacks due to the lack of slippage protections	Security Issue	Confirmed
14	-	Add allowance revocation logic	Recommendation	Fixed
15	-	Revise the validation order in the function <code>_bondingCurveSell()</code>	Recommendation	Fixed
16	-	Remove redundant code	Recommendation	Fixed
17	-	Fix typo	Recommendation	Confirmed

18	-	Add a tax validation check in the function <code>setTaxConfig()</code>	Recommendation	Fixed
19	-	Potential centralization risks	Note	-
20	-	Disabled two-step token creation and permitted token name/symbol duplication	Note	-
21	-	Dividend withdrawal failure handling mechanism	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Potential skipping of token distribution in the function `distributeTokens()`

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `SaleForge`, the function `distributeTokens()` is designed to distribute base tokens to pre-sale participants, using the input `limit` to control batch processing. However, the logic is flawed because it does not validate whether the input `limit` is zero. Specifically, if `limit` equals zero, the loop is skipped, yet the storage variable `lastProcessedIndex` is still incremented. As a result, this flaw may cause the function to skip distribution for certain users, preventing them from receiving their purchased tokens.

```

1015     for (uint256 i = startIndex; i < totalCount && (i - startIndex) < limit; i++) {
1016         (address participant, uint256 packed) = participantMap.at(i);
1017         ParticipantInfo memory info = _unpackParticipantInfo(packed);
1018
1019         // Always update the last processed index as we iterate
1020         newLastIndex = i;
1021
1022         if (!info.claimed && info.participation > 0) {
1023             // Calculate and emit distribution in one go to save stack space
1024             uint256 tokenAmount = (uint256(info.participation) * immutable_.economics.tokens) /
                immutable_.raiseAmt;
1025
1026             // Update claim status
1027             info.claimed = true;
1028             _setParticipantInfo(token, participant, info);
1029
1030             // transfer tokens
1031             SafeTransferLib.safeTransfer(token, participant, tokenAmount);
1032
1033             // Emit individual distribution event
1034             emit FlapSaleTokensDistributed(token, participant, tokenAmount);
1035         }
1036     }

```

```
1037
1038     // Update the last processed index in storage
1039     sales[token].lastProcessedIndex = uint64(newLastIndex + 1);
```

Listing 2.1: src/FlapSale/SaleForge.sol

Impact Users may not receive their purchased base tokens from the pre-sale.

Suggestion Revise the logic accordingly.

2.1.2 Lack of token transfer logic in function `claimTokens()`

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `SaleForge`, the function `claimTokens()` is intended to allow pre-sale participants to claim their tokens. However, the function only updates the user's claim status and does not execute the actual token transfer, which may result in participants being unable to receive their tokens.

```
951     // Transfer tokens (would need actual token transfer here)
952     // This is a stub since finalizeSale is also a stub
```

Listing 2.2: src/FlapSale/SaleForge.sol

Impact Participants lose the tokens they purchased in the pre-sale.

Suggestion Add the token transfer logic.

2.1.3 Significant price deviation between bonding curve and DEX launch price

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The protocol supports multiple `CurveType` and `DexThreshold` combinations to determine the DEX launch price of a meme token. While the bonding curve introduces virtual liquidity and is designed to be deeper than the initial DEX liquidity, the protocol allows up to a 20% deviation between curve price and DEX launch price.

However, testing reveals that several `CurveType` and `DexThreshold` configurations produce price discrepancies that significantly exceed this intended threshold. These deviations arise primarily because the remaining base-token reserve at the moment of DEX launch is insufficient to support the intended price, causing the actual DEX launch price to diverge significantly from the bonding-curve purchase price. Specifically:

Curve = CURVE_RH_28D25_108002126, DexThreshold = NINETY_FIVE_PERCENT, Ratio(sport_price_in_dex/sport_price_in_curve) = 2.709418

Curve = CURVE_RH_14981_108002125, DexThreshold = NINETY_FIVE_PERCENT, Ratio(sport_price_in_dex/sport_price_in_curve) = 2.709418

```
Curve = CURVE_RH_TOSHI_MORPH_2ETH, DexThreshold = NINETY_FIVE_PERCENT, Ratio(sport_price_in_dex/sport_price_in_curve) = 2.695212
Curve = CURVE_RH_TOSHI, DexThreshold = NINETY_FIVE_PERCENT, Ratio(sport_price_in_dex/sport_price_in_curve) = 2.695212
Curve = CURVE_RH_BGB, DexThreshold = NINETY_FIVE_PERCENT, Ratio(sport_price_in_dex/sport_price_in_curve) = 2.695212
Curve = CURVE_RH_BNB, DexThreshold = NINETY_FIVE_PERCENT, Ratio(sport_price_in_dex/sport_price_in_curve) = 2.695212
Curve = CURVE_RH_USD, DexThreshold = NINETY_FIVE_PERCENT, Ratio(sport_price_in_dex/sport_price_in_curve) = 2.695212
Curve = CURVE_RH_MONAD, DexThreshold = NINETY_FIVE_PERCENT, Ratio(sport_price_in_dex/sport_price_in_curve) = 2.695212
```

Such discrepancies create clear arbitrage opportunities for malicious users, potentially harming regular participants and impacting the protocol's overall market dynamics.

Impact The large deviation between the [DEX](#) launch price and the final bonding curve price enables arbitrage, negatively affecting other token holders.

Suggestion Revise the logic accordingly.

2.1.4 Lack of relative privileged function implementation in contract [Portal](#)

Severity Medium

Status Confirmed

Introduced by [Version 1](#)

Description The [Dividend](#) and [TaxProcessor](#) contracts assign their ownership to the [Portal](#) during initialization, leaving [Portal](#) as the sole entity authorized to execute their owner-restricted functions. However, [Portal](#) does not implement any logic to invoke these functions, making all owner-gated operations unreachable. As a result, configuration updates, emergency withdrawals, and administrative controls in both contracts can not be executed once deployed.

```
165 function excludeAddress(address addr) external onlyOwner {
166     require(!excludedFromDividends[addr], "Dividend: already excluded");
167     excludedFromDividends[addr] = true;
168
169     // Reset their share to 0
170     if (userInfo[addr].share > 0) {
171         _setShare(addr, 0);
172     }
173
174     emit AddressExcluded(addr);
175 }
```

Listing 2.3: src/Tax/Dividend.sol

```
179 function setMinimumShareBalance(uint256 newMinimumShareBalance) external onlyOwner {
180     minimumShareBalance = newMinimumShareBalance;
181 }
```

Listing 2.4: src/Tax/Dividend.sol

```

317 function emergencyWithdraw(address token, uint256 amount, address to) external onlyOwner {
318     require(to != address(0), "Dividend: invalid recipient");
319
320     if (token == address(0)) {
321         // Withdraw native ETH
322         uint256 balance = address(this).balance;
323         uint256 withdrawAmount = amount == 0 ? balance : amount;
324         require(withdrawAmount <= balance, "Dividend: insufficient ETH balance");
325
326         (bool success,) = payable(to).call{value: withdrawAmount}("");
327         require(success, "Dividend: ETH transfer failed");
328     } else {
329         // Withdraw ERC20 token
330         IERC20 tokenContract = IERC20(token);
331         uint256 balance = tokenContract.balanceOf(address(this));
332         uint256 withdrawAmount = amount == 0 ? balance : amount;
333         require(withdrawAmount <= balance, "Dividend: insufficient token balance");
334
335         tokenContract.safeTransfer(to, withdrawAmount);
336     }
337 }

```

Listing 2.5: src/Tax/Dividend.sol

```

382 function setReceivers(address feeReceiver_, address marketAddress_, address dividendAddress_)
383     external onlyOwner {
384     require(feeReceiver_ != address(0), "TaxProcessor: zero fee receiver");
385     feeReceiver = feeReceiver_;
386
387     // Only set market and dividend addresses if they will be used (bps > 0)
388     PackedFeeConfig memory config = feeConfig;
389
390     if (config.marketBps > 0) {
391         require(marketAddress_ != address(0), "TaxProcessor: zero market address");
392         marketAddress = marketAddress_;
393     }
394
395     if (config.dividendBps > 0) {
396         require(dividendAddress_ != address(0), "TaxProcessor: zero dividend address");
397         dividendAddress = dividendAddress_;
398     }
399 }

```

Listing 2.6: src/Tax/TaxProcessor.sol

```

401 function setTaxConfig(uint16 feeRate_, uint16 marketBps_, uint16 deflationBps_, uint16 lpBps_,
402     uint16 dividendBps_)
403     external
404     onlyOwner
405 {

```

```
405 // Validate fee rate and percentages
406 require(feeRate_ <= 10000, "TaxProcessor: feeRate must be <= 10000");
407 uint256 totalBps = uint256(marketBps_) + uint256(deflationBps_) + uint256(lpBps_) + uint256
    (dividendBps_);
408 require(totalBps == 10000, "TaxProcessor: distribution bps must sum to 10000");
409
410 // Update packed config
411 PackedFeeConfig memory config = feeConfig;
412 config.feeRate = feeRate_;
413 config.marketBps = marketBps_;
414 config.deflationBps = deflationBps_;
415 config.lpBps = lpBps_;
416 config.dividendBps = dividendBps_;
417 feeConfig = config;
418 }
```

Listing 2.7: src/Tax/TaxProcessor.sol

```
506 function withdrawAll(address token, address to) external onlyOwner {
507     require(to != address(0), "TaxProcessor: zero address");
508
509     if (token == address(0)) {
510         // Default to withdraw quote token or native ETH
511         PackedFeeConfig memory config = feeConfig;
512         if (config.isWeth) {
513             // If quote token is WETH, withdraw native ETH
514             uint256 balance = address(this).balance;
515             if (balance > 0) {
516                 (bool success,) = payable(to).call{value: balance}("");
517                 require(success, "TaxProcessor: ETH transfer failed");
518             }
519         } else {
520             // Withdraw quote token
521             address quoteTokenAddr = _getQuoteToken();
522             uint256 balance = IERC20(quoteTokenAddr).balanceOf(address(this));
523             if (balance > 0) {
524                 IERC20(quoteTokenAddr).safeTransfer(to, balance);
525             }
526         }
527     } else {
528         // Withdraw specific ERC20 token
529         uint256 balance = IERC20(token).balanceOf(address(this));
530         if (balance > 0) {
531             IERC20(token).safeTransfer(to, balance);
532         }
533     }
534 }
```

Listing 2.8: src/Tax/TaxProcessor.sol

Impact The owner restricted functions become unusable, resulting in DoS issue.

Suggestion Add related functions to the contract [Portal](#).

Feedback from the project Since the Portal is upgradeable, we would like to implement these in the future when needed. In the ideal case, we don't need to call these methods.

2.1.5 Potential DoS due to the improper native token swap logic

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract [PortalTradeV2](#), the function `_swapExactInputV3()` handles native token swaps when users purchase tokens with non-native quote tokens. However, the design is problematic due to improper path generation when the configured quoteToken is `WETH`. Specifically, the swap path is generated as `[WETH, WETH]`, which causes transaction reverts in the following swap logic. As a result, this design results in a denial-of-service condition, leading to failed transactions and disrupted trading functionality.

```
177     expectedQuoteToken = state.quoteTokenAddress;
178
179     // 4. determine if quoteSwapRequired
180     // If the expected quote token is not the same as the state quote token, we need to swap
181     quoteSwapRequired = expectedQuoteToken != quoteToken;
```

Listing 2.9: src/PortalTradeV2.sol

```
78     if (isBuy) {
79         uint256 inputAmount = params.inputAmount;
80         if (quoteSwapRequired) {
81             // Swap native to expected quote token
82             inputAmount = _nativeToQuoteSwap(expectedQuoteToken, params.inputAmount);
83         }
84         uint256 quoteRefund;
85         (outputAmount, quoteRefund) = _bondingCurveBuy(baseToken, msg.sender, inputAmount,
86             params.minOutputAmount);
87
88         // Handle refund if any
89         if (quoteRefund > 0) {
90             if (quoteSwapRequired) {
91                 // Swap excess quote token back to native gas token
92                 /*uint256 nativeRefund =*/
93                 _quoteToNativeSwap(expectedQuoteToken, quoteRefund, msg.sender);
94             } else {
95                 // Direct refund of quote token
96                 _payOut(expectedQuoteToken, quoteRefund, msg.sender);
97             }
98         } else {
99             if (quoteSwapRequired) {
100                 // inf slippage on bonding curve
101                 outputAmount = _bondingCurveSell(baseToken, address(this), params.inputAmount, 0);
102                 // Swap quote token to native
103                 outputAmount = _quoteToNativeSwap(expectedQuoteToken, outputAmount, msg.sender);
```

```
104         // check slippage after quote to native swap
105         if (outputAmount < params.minOutputAmount) {
106             revert SlippageTooHigh(outputAmount, params.minOutputAmount);
107         }
108     } else {
109         outputAmount = _bondingCurveSell(baseToken, msg.sender, params.inputAmount, params.
            minOutputAmount);
110     }
111 }
```

Listing 2.10: src/PortalTradeV2.sol

```
193     amountOut = ISwapRouter(smartRouter).swapExactTokensForTokens(
194         0, // Use router's balance
195         amountOutMin,
196         path,
197         to
198     );
```

Listing 2.11: src/MultiDexRouter.sol

Impact This design results in a DoS condition, leading to failed transactions and disrupted trading functionality.

Suggestion Add proper logic while trading with the native token.

2.1.6 Potential DoS due to incorrect validation in function `_participate()`

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The `_participate()` function enforces a fixed minimum purchase requirement based on `immutable_.minBuy`. When the remaining sale capacity drops below this threshold, any further participation attempts are strictly rejected because the validation logic still requires the full minimum amount. In normal operation, this can leave a residual capacity smaller than the minimum buy value, making all subsequent participation attempts revert and preventing the sale from completing its final allocation.

```
898     if (acceptedAmount < immutable_.minBuy || newTotal > immutable_.maxBuy) {
899         revert InvalidAmount();
900     }
```

Listing 2.12: src/FlapSale/SaleForge.sol

Impact A residual capacity below the minimum threshold can block further participation and prevent the sale from reaching its target.

Suggestion Enforce the minimum-purchase requirement only when the remaining capacity is large enough to satisfy it.

2.1.7 Incorrect fee calculation in the function `_previewBondingCurveBuy()`

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `PortalTradeV2`, the function `_previewBondingCurveBuy()` computes fees based on the input `quoteToken` amount. However, the design is problematic due to charging fees on the entire input rather than the actual purchased amount. Specifically, when a user submits an amount exceeding the purchase cap, the excess `quoteToken` is still subject to fees. As a result, this approach leads to overcharging users, causing unintended financial losses.

```
250         // Calculate fee on the input amount
251         feeQuote = (quoteIn * _getFeeRate(msg.sender, token, true)) / 10_000;
252         uint256 netQuote = quoteIn - feeQuote;
```

Listing 2.13: `src/PortalTradeV2.sol`

Impact Users may be overcharged fees, resulting in unintended loss of funds.

Suggestion Calculate the fee based on the actual purchased amount.

2.1.8 Lack of native token refund mechanism

Severity Low

Status Confirmed

Introduced by [Version 1](#)

Description `PortalTradeV2` allows users to pay with native tokens when executing swaps. The contract only checks that the native token sent is not less than the parameter-specified input amount, while the swap logic consumes only that specified amount. Any excess native token sent remains in the contract and is not refunded. A similar issue also exists in the base token creation flow. Specifically, when the quote token is non-native and taxation applies, creators may overpay native tokens for wrapped token conversion, and excess amounts are also not refunded.

```
462     function _payIn(address inputToken, uint256 inputAmount, bytes memory permitData) internal {
463         PermitHelper.transferWithPermit(inputToken, inputAmount, permitData, address(this), msg.
            sender, msg.value);
464     }
```

Listing 2.14: `src/PortalTradeV2.sol`

```
67     function _swapExactInputV3(ExactInputV3Params memory params) internal returns (uint256
        outputAmount) {
68         (address baseToken, address expectedQuoteToken, bool isBuy, bool quoteSwapRequired) =
69             _getRoute(params.inputToken, params.outputToken);
70
71         // Handle extension on trade if necessary
72         PackedTokenStateV2 memory state = _getTokenState(baseToken);
```

```

73     _handleExtensionOnTrade(params.extensionData, msg.sender, state.extensionID, baseToken);
74
75     // Payment logic
76     _payIn(params.inputToken, params.inputAmount, params.permitData);

```

Listing 2.15: src/PortalTradeV2.sol

```

316     if (params.taxRate > 0) {
317         // extra post creation jobs for tax tokens if the quote token is not native
318         if (params.quoteToken != address(0)) {
319             if (msg.value < EXTRA_WETH_FOR_TAX_TOKEN_WITH_ERC20_QUOTE) {
320                 revert InsufficientValueForTaxTokenCreation(
321                     EXTRA_WETH_FOR_TAX_TOKEN_WITH_ERC20_QUOTE, msg.value);
322             }
323
324             // convert 1gwei to WETH
325             WETH(payable(WETH_ADDRESS)).deposit{value:
326                 EXTRA_WETH_FOR_TAX_TOKEN_WITH_ERC20_QUOTE}();
327             // transfer it to the token contract address
328             WETH(payable(WETH_ADDRESS)).transfer(token,
329                 EXTRA_WETH_FOR_TAX_TOKEN_WITH_ERC20_QUOTE);
330         }
331     }

```

Listing 2.16: src/PortalTokenLauncher.sol

Impact Users may unintentionally lose native tokens due to unrefunded excess payments.

Suggestion Apply refund mechanism.

Feedback from the project To avoid introducing unexpected external call, we decided not to fix this.

2.1.9 Lack of check in function `stageNewTokenV5()`

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The function `stageNewTokenV5()` does not verify whether the `dexThresh` parameter supplied by the user falls within the valid bounds defined by the protocol. In its current form, function `stageNewTokenV5()` derives the token address from the provided parameters and constructs the corresponding token state, which is then stored on-chain.

However, the function does not invoke function `_validateDexThresholdType()` to enforce correctness of the `dexThresh` value. As a result, a user may submit an invalid or unsuitable `dexThresh`, potentially leading to incorrect or unintended behavior in the token's lifecycle.

```

674     function stageNewTokenV5(StageNewTokenV5Params calldata params) external override returns (
675         address token) {

```

Listing 2.17: src/PortalTokenLauncher.sol

Impact Invalid or out of range `dexThresh` values may lead to incorrect lifecycle transitions, causing tokens to malfunction or become stuck between the bonding curve and DEX-launch phases.

Suggestion Invoke the function `_validateDexThresholdType()` in the function `stageNewTokenV5()` to ensure `dexThresh` is within the permitted range and revert on invalid input.

2.1.10 Potential DoS while handling non-compliant ERC20 tokens

Severity Low

Status Confirmed

Introduced by Version 1

Description In the contract, the function `transferFrom()` utilizes the `ITokenV2` interface which assumes `ERC20` tokens return `boolean` values. However, the design is problematic for non-compliant tokens (e.g., `USDT`) that do not return `booleans` and will revert transactions.

```
50         bool ok = ITokenV2(token).transferFrom(sender, recipient, amount);
51         if (!ok) {
52             revert TransferFromFailed(token, sender, amount);
53         }
```

Listing 2.18: `src/libraries/PermitHelper.sol`

Impact Transactions involving non-compliant tokens may revert and disrupt normal operation.

Suggestion Adopt a compatibility-safe transfer method such as `TransferHelper.safeTransferFrom()` to support tokens without `boolean` return values.

Feedback from the project Confirmed. `USDT` on the BSC does not have this issue.

2.1.11 Potential overcharge in the function `createSale()`

Severity Low

Status Confirmed

Introduced by Version 1

Description In the contract `SaleForge`, the function `createSale()` is intended to charge a specific creation fee `FSALE_CREATION_FEE_BPS`. However, the current implementation forwards the entire `msg.value` to `FSALE_FEE_RECEIVER`,

```
424         // Transfer creation fee to fee receiver
425         payable(FSALE_FEE_RECEIVER).sendValue(msg.value);
```

Listing 2.19: `src/FlapSale/SaleForge.sol`

Impact Users may unintentionally lose native tokens due to overpaid fees.

Suggestion Charge fees according to `FSALE_CREATION_FEE_BPS`.

Feedback from the project Confirmed, this overcharge is by design for simplification.

2.1.12 Potential permanent lock of residual tax tokens in function `_liquidateTax()`

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In contract `FlapTaxTokenV2`, the function `_liquidateTax()` sets the pool state to `TaxFree` before the contract confirms that accumulated tax tokens are successfully processed. The actual swap and distribution occur in function `_processTax()`, which invokes the external `TaxProcessor` contract inside a `try/catch` and does not revert on failure. If that external call fails, the contract remains in `TaxFree` while the tax token balance stays in the contract. Subsequent invocations skip processing because the function only acts when the state is `TaxEnforced` or `TaxEnforcedAntiFarmer`, so normally occurring failures can leave a residual tax balance permanently inaccessible.

```

390 }
391
392 /// @notice Internal helper to process tax tokens
393 /// @param taxAmount The amount of tax tokens to process
394 function _processTax(uint256 taxAmount) internal returns (bool) {
395     if (taxAmount == 0) return true;
396
397     // Approve TaxProcessor to pull tokens
398     if (allowance(address(this), taxProcessor) < taxAmount) {
399         _approve(address(this), taxProcessor, type(uint256).max);
400     }
401
402     // Call TaxProcessor.processTaxTokens with total tax amount
403     // The TaxProcessor will read configuration from this contract (msg.sender)
404     // and handle fee computation, remainder splitting, deflation burning, and distribution
405     try ITaxProcessor(taxProcessor).processTaxTokens(taxAmount) {
406         // Success
407     } catch (bytes memory reason) {
408         emit TaxLiquidationError(reason);
409     }

```

Listing 2.20: `src/Tax/FlapTaxTokenV2.sol`

Impact The residual tax tokens become irretrievably stuck in the contract.

Suggestion Revise the logic to ensure that function `_processTax()` executes successfully before updating the state to `TaxFree`.

2.1.13 Potential sandwich attacks due to the lack of slippage protections

Severity Low

Status Confirmed

Introduced by [Version 1](#)

Description In the contract `FlapTaxToken`, the function `_liquidateTax()` is designed to liquidate the accrued tax during token transfers. When tax reaches its threshold amount or the

state is updated to `PoolState.TaxFree`, all accrued tax tokens (i.e., `taxAmount`) are swapped for `QUOTE_TOKEN` via the contract `PCS_V2_ROUTER`. However, there is a lack of slippage protection for the swap, allowing attackers to gain profits by performing sandwich attacks.

The same issue also exists in the contracts `TaxProcessor`.

```
431 function _swapTokensForQuote(uint256 amount) internal returns (uint256 quoteReceived) {
432     // Approve router
433     IERC20(taxToken).safeApprove(router, 0);
434     IERC20(taxToken).safeApprove(router, amount);
435
436     address quoteTokenAddr = _getQuoteToken();
437     address[] memory path = new address[](2);
438     path[0] = taxToken;
439     path[1] = quoteTokenAddr;
440
441     uint256 quoteBefore = IERC20(quoteTokenAddr).balanceOf(address(this));
442
443     IUniswapRouter02(router).swapExactTokensForTokensSupportingFeeOnTransferTokens(
444         amount,
445         0, // Accept any amount
446         path,
447         address(this),
448         block.timestamp
449     );
450     quoteReceived = IERC20(quoteTokenAddr).balanceOf(address(this)) - quoteBefore;
451
452     return quoteReceived;
453 }
```

Listing 2.21: `src/Tax/TaxProcessor.sol`

Impact The tax liquidation process may suffer sandwich attacks due to the lack of slippage protections.

Suggestion Add slippage protections.

Feedback from the project The project stated that they are aware of this issue and would mitigate the issue by setting reasonable values for the variables `taxRate` and `liquidationThreshold`.

2.2 Recommendation

2.2.1 Add allowance revocation logic

Status Fixed in `Version 2`

Introduced by `Version 1`

Description The function `_quoteToNativeSwap()` grants token allowances to `MULTI_DEX_ROUTER` to facilitate `swap` operations. However, the current implementation does not revoke these allowances after the `swap` operation completes, leaving residual allowances that could potentially

be exploited by malicious actors or create unintended security risks. It is recommended to implement allowance revocation logic immediately after each `swap` operation to ensure no residual allowances remain.

```
557     ITokenV2(quoteToken).approve(address(MULTI_DEX_ROUTER), quoteAmt);
```

Listing 2.22: src/PortalTradeV2.sol

Suggestion Add explicit allowance revocation by invoking function `approve(MULTI_DEX_ROUTER, 0)`.

2.2.2 Revise the validation order in the function `_bondingCurveSell()`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the function `_bondingCurveSell()`, the contract first invokes function `_previewBondingCurveSell()` to calculate the `quoteOut` value based on the provided `inputAmount`. Subsequently, the contract performs slippage protection and a non-zero check. This sequence becomes problematic when `quoteOut` equals to zero, as zero will always be less than any positive `minOutputAmount`, causing the function to revert with `SlippageTooHigh` error instead of the more appropriate `AmountTooSmall` error. It is recommended to adjust the validation order to ensure more accurate error reporting.

```
396     if (quoteOut < minOutputAmount) revert SlippageTooHigh(quoteOut, minOutputAmount);
397     if (quoteOut == 0) revert AmountTooSmall(inputAmount);
```

Listing 2.23: src/PortalTradeV2.sol

Suggestion Revise the validation order.

2.2.3 Remove redundant code

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description There are several unused variables, events, functions. It is recommended to remove them for better code readability. Specifically, the following code should be removed or revised.

```
375     function _chargeCreationFee(uint256 fee) internal {
376         if (msg.value < fee) {
377             revert InsufficientFee(fee, msg.value);
378         }
379
380         if (fee > 0) {
381             AddressUpgradeable.sendValue(payable(FEE_RECEIVER), fee);
382         }
383     }
```

Listing 2.24: src/PortalTokenLauncher.sol

```
566 function _isTokenStaged(address token) internal view returns (bool) {
567     PackedTokenStateV2 memory state = _getTokenState(token);
568     return state.status == TokenStatus.Staged;
569 }
```

Listing 2.25: src/PortalTokenLauncher.sol

In the contract `PortalTokenLauncher`, the function `commitNewTokenV5()` increases `_nonce` twice.

```
773 uint256 tokenNonce = _nonce++;
```

Listing 2.26: src/PortalTokenLauncher.sol

```
164 tokenNonce = _nonce++;
```

Listing 2.27: src/PortalTokenLauncher.sol

```
1198 tokenNonce = _nonce++;
```

Listing 2.28: src/PortalTokenLauncher.sol

In the tax token's transfer logic, the function `_liquidateTax()` is invoked to determine the current state of the token. If the accumulated tax token fees exceed a predefined threshold, the function transfers these tax tokens to the `taxProcessor` contract for further processing. Since the tax token contract does not interact directly with the `v2Router` for transfers, the invoke to `_approve()` inside this function is redundant.

```
329 if (allowance(address(this), v2Router) < taxAmount) {
330     _approve(address(this), v2Router, type(uint256).max);
331 }
```

Listing 2.29: src/Tax/FlapTaxTokenV2.sol

Suggestion Remove the redundant code.

2.2.4 Fix typo

Status Confirmed

Introduced by Version 1

Description In the contract `PortalUniV2Migrator`, there's a typo in the function `luanchToDEX()`, the correct function name should be `launchToDEX()`.

```
41 function luanchToDEX(address token) external payable {
```

Listing 2.30: src/PortalUniV2Migrator.sol

Suggestion Revise the typo.

Feedback from the project We will not fix this, due to a lot of dependencies and external scripts.

2.2.5 Add a tax validation check in the function `setTaxConfig()`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The function `setTaxConfig()` allows the owner to update tax distribution percentages without verifying whether the corresponding recipient addresses (e.g., `marketAddress` and `dividendAddress`) are valid. If a component was initially disabled and thus has a zero address, subsequently assigning it a positive BPS share will result in funds being directed to the zero address.

```
400  /// @notice Owner can update tax configuration
401  function setTaxConfig(uint16 feeRate_, uint16 marketBps_, uint16 deflationBps_, uint16 lpBps_,
    uint16 dividendBps_)
402      external
403      onlyOwner
404  {
405      // Validate fee rate and percentages
406      require(feeRate_ <= 10000, "TaxProcessor: feeRate must be <= 10000");
407      uint256 totalBps = uint256(marketBps_) + uint256(deflationBps_) + uint256(lpBps_) + uint256
    (dividendBps_);
408      require(totalBps == 10000, "TaxProcessor: distribution bps must sum to 10000");
409
410      // Update packed config
411      PackedFeeConfig memory config = feeConfig;
412      config.feeRate = feeRate_;
413      config.marketBps = marketBps_;
414      config.deflationBps = deflationBps_;
415      config.lpBps = lpBps_;
416      config.dividendBps = dividendBps_;
417      feeConfig = config;
418  }
```

Listing 2.31: `src/Tax/TaxProcessor.sol`

Suggestion Revise the logic to ensure that any category allocated a non-zero percentage has a properly initialized address.

2.3 Note

2.3.1 Potential centralization risks

Introduced by [Version 1](#)

Description In this project, several privileged roles (e.g., `GUARDIAN_ROLE` and `DEFAULT_ADMIN_ROLE`) can conduct sensitive operations, which introduces potential centralization risks. For example, the privileged roles can set bit flags and halt the protocol. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

Feedback from the project All privileged operations are executed through a 3 of 4 Gnosis Safe wallet.

2.3.2 Disabled two-step token creation and permitted token name/symbol duplication

Introduced by [Version 1](#)

Description The staged token-creation flow, composed of [stageNewTokenV5](#) and [commitNewTokenV5](#), relied on a deterministic address derived from a creator-supplied [salt](#), which allowed a malicious actor to preempt the [salt](#) via front-running and block the intended deployment. This pipeline has been currently disabled, leaving the underlying code path unused and no longer relevant to active token-creation logic.

The protocol also intentionally allows non-unique token names and symbols, consistent with the project team's design choice to permit collisions within the ecosystem.

Feedback from the project We are considering charging an extra fee to avoid vanity mining.

2.3.3 Dividend withdrawal failure handling mechanism

Introduced by [Version 1](#)

Description The function [_withdrawDividendOfUser\(\)](#) implements a design pattern where failed token transfers do not revert the transaction but instead emit a [DividendWithdrawalFailed](#) event. In such cases, the user's internal accounting state ([rewardDebt](#) and [pendingBalance](#)) is updated as if the withdrawal succeeded, effectively marking the dividends as claimed within the contract's logic despite the funds remaining in the contract balance. While this prevents a single user's failure from blocking batch operations, it requires admin manual intervention via the [emergencyWithdraw\(\)](#) function to recover these specific funds.

Feedback from the project Since Portal is upgradeable, we may support new fallback logics in the future. Ideally, we will not help recover the locked funds for most of the case. The user should take care of it.

